

Table of Contents

Chapter 2: Computational Thinking & Algorithms	4
Introduction	4
2.1 Introduction to Computational Thinking	4
The Hook (Story Mode)	4
The Explanation	5
Real-Life Examples of Computational Thinking	5
The Practical Walkthrough	5
Interactive Stop-Point	6
Quick Recap	6
2.2 Problem Solving Using Computational Thinking	6
The Hook (Story Mode)	6
The Explanation	6
The Practical Walkthrough	7
Interactive Stop-Point	8
Quick Recap	8
2.3 Logic as a Knowledge Representation Framework	8
The Hook (Story Mode)	8
The Explanation	9
The Practical Walkthrough	9
Interactive Stop-Point	9
Quick Recap	9
2.4 Propositional Logic	10
The Hook (Story Mode)	10
The Explanation	10
The Practical Walkthrough	11
Interactive Stop-Point	12
Quick Recap	12
2.5 Truth Tables	12
The Hook (Story Mode)	12
The Explanation	12
The Practical Walkthrough: Building a Full Truth Table for AND, OR, and NOT	13
The Practical Walkthrough: Truth Table for IMPLIES ($\rightarrow$) and BICONDITIONAL ($\leftrightarrow$)	13
Interactive Stop-Point	14
Quick Recap	14

2.6 Propositional Equivalence	14
The Hook (Story Mode)	14
The Explanation.....	14
The Practical Walkthrough: Proving De Morgan's Law with a Truth Table	15
Interactive Stop-Point	15
Quick Recap	15
2.7 Propositional Satisfiability.....	16
The Hook (Story Mode)	16
The Explanation.....	16
The Practical Walkthrough: Testing Satisfiability of $(P \vee Q) \wedge (\neg P \vee \neg Q)$	16
Interactive Stop-Point	17
Quick Recap	17
2.8 Predicate Logic.....	17
The Hook (Story Mode)	17
The Explanation.....	18
The Practical Walkthrough: Translating Predicates	19
Interactive Stop-Point	19
Quick Recap	19
2.9 Quantifiers in Predicate Logic	19
The Hook (Story Mode)	19
The Explanation.....	20
The Practical Walkthrough: Quantifier Order Matters!	20
Interactive Stop-Point	21
Quick Recap	21
2.10 Applying Predicate Logic to Real-World Problems.....	21
The Hook (Story Mode)	21
The Explanation.....	21
The Practical Walkthrough: Modeling a Library System	22
Interactive Stop-Point	23
Quick Recap	23
2.11 Logic-Based Reasoning and Inference.....	23
The Hook (Story Mode)	23
The Explanation.....	23
The Practical Walkthrough: A Security Lockout Decision Using Deduction.....	24
Interactive Stop-Point	25
Quick Recap	25
Chapter Summary	26

End-of-Chapter Exercise	27
Multiple Choice Questions	27
Short Questions	27
Long Questions	28

Chapter 2: Computational Thinking & Algorithms

A CS50-style guide to thinking like a problem solver — and like a machine

Introduction

Every great programmer, before writing a single line of code, does something much more important: they *think*. Not randomly — systematically. This chapter teaches you two closely related superpowers.

First, **computational thinking** — a structured way to break down any problem, from planning a birthday party to designing a search engine, into pieces small enough to solve one at a time.

Second, **logic** — the formal language that lets you describe true and false with total precision, so that a rule you write is never ambiguous, whether it's read by a human, a computer chip, or an entire security system.

By the end of this chapter, you'll be able to translate messy, ambiguous English sentences ("free shipping if you spend \$50 and you're a member, or it's your first order") into crisp, testable logical expressions — the exact same skill that powers every **if** statement, every database search, and every AI reasoning engine on Earth. Let's begin.

2.1 Introduction to Computational Thinking

The Hook (Story Mode)

In 1847, a self-taught English mathematician named George Boole published a small book with a huge idea: that human thought — reasoning, logic, true and false — could be captured using algebra, with just two values: 1 and 0. At the time, this seemed like a strange, purely academic exercise. Nobody could have guessed that, roughly a century later, every single transistor inside every computer chip on the planet would be built directly on Boole's "Boolean algebra." Every time you unlock your phone, you're using George Boole's 175-year-old idea, running billions of times per second.

The Explanation

Computational thinking is a way of thinking that helps you solve problems clearly and logically — not just with computers, but in daily life. It's not about typing code; it's about the *thought process* that happens before any code gets written.

Computational thinking has four core habits:

- **Decomposition** — breaking a big problem into smaller, manageable parts.
- **Pattern Recognition** — noticing similarities between problems you've already solved.
- **Abstraction** — ignoring unimportant details and focusing on what actually matters.
- **Algorithm Design** — creating clear, step-by-step instructions to reach a solution.

Computational Thinking in Daily Life

- ```

1. Decomposition --> Break the problem down
2. Pattern Recognition --> Find similarities/patterns
3. Abstraction --> Focus on what matters
4. Algorithm Design --> Create step-by-step instructions
```

*Link:* This is not a "computer science only" skill. Scientists use it to design experiments. Chefs use it to plan a five-course meal. Engineers use it to build bridges. You already use it — this chapter just gives it a name and a method.

## Real-Life Examples of Computational Thinking

- **Making a daily timetable** — deciding what to do and when, in a logical order.
- **Cooking a recipe** — following exact step-by-step instructions to get a reliable result.
- **Planning a trip** — breaking a big goal ("visit Lahore") into smaller tasks (book transport, book hotel, plan sightseeing).
- **Sorting books** by subject or size — this is pattern recognition in action.
- **Fixing a phone problem** — using logical elimination ("is it the battery? the software? the charger?").
- **Managing money** — planning a budget requires decomposition and decision-making.
- **Playing strategy games** — chess or strategy video games require thinking several steps ahead.

## The Practical Walkthrough

Let's apply all four habits to one real task: **"Plan a class trip to a science museum."**

| Habit               | Applied to the class trip                                                                                                      |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Decomposition       | Break into: transport, tickets, food, schedule, permission slips                                                               |
| Pattern Recognition | "This is similar to how we planned last year's sports day"                                                                     |
| Abstraction         | Ignore irrelevant details (what color the bus is); focus on capacity, cost, and timing                                         |
| Algorithm Design    | Step 1: Get budget approved → Step 2: Book bus → Step 3: Buy tickets → Step 4: Confirm headcount → Step 5: Distribute schedule |

**What just happened?** A vague, overwhelming goal ("plan a trip") became five clear, doable steps — this exact process is what programmers do every single day before writing any code.

## Interactive Stop-Point

**Pause & Think:** Pick any everyday activity you did this week (studying for a test, organizing your room, planning a workout). Identify which of the four computational thinking habits (decomposition, pattern recognition, abstraction, algorithm design) you used, even without realizing it.

## Quick Recap

Computational thinking is a universal problem-solving method built on four habits — decomposition, pattern recognition, abstraction, and algorithm design — used by everyone from chefs to computer scientists, not just programmers.

---

## 2.2 Problem Solving Using Computational Thinking

### The Hook (Story Mode)

Imagine handing a computer a vague instruction like "make things better." It would have no idea what to do — computers need painfully precise instructions. Interestingly, humans struggle with vague problems too. Give someone the task "fix the printer" with no other information, and they'll likely freeze, unsure where to start. Computational thinking exists to solve exactly this: turning vague chaos into clear, solvable steps — for humans and computers alike.

### The Explanation

Problem solving using computational thinking follows four connected stages, each building on the last.

#### 1. Identifying a Problem Clearly

Before solving anything, you must understand exactly what's being asked. What information do you already have? What result are you trying to reach? Ignore irrelevant details. A well-understood problem is already halfway solved.

## 2. Breaking a Problem into Smaller Parts (Decomposition)

A big, scary problem becomes manageable once split into smaller pieces. Each piece can be tackled on its own, in order, without feeling overwhelmed by the whole.

## 3. Creating Step-by-Step Solutions

The ordered steps used to solve a problem are called an **algorithm**. Each step must be simple, unambiguous, and in the correct order — this is exactly the discipline computers require, and exactly the discipline this chapter builds in you.

```
Step-by-Step Algorithm Example: Making Tea
```

```

```

1. Boil water
2. Add tea leaves
3. Let it steep
4. Add milk or lemon
5. Serve

## 4. Evaluating and Improving Solutions

Once a solution exists, test it. Does it actually work? Can it be made simpler or faster? Evaluation catches mistakes and drives improvement — this is exactly why software gets "updates" after release.

*Link:* This four-stage cycle is literally what a programmer does building an app feature: understand the request → break it into functions → write the step-by-step code → test and refine it.

## The Practical Walkthrough

Let's solve a real problem end-to-end: **"My WhatsApp keeps freezing."**

| Stage                    | Applied                                                                                                            |
|--------------------------|--------------------------------------------------------------------------------------------------------------------|
| 1. Identify clearly      | What exactly happens? Does it freeze on all chats, or just one with lots of images?                                |
| 2. Decompose             | Possible causes: (a) low phone storage, (b) outdated app version, (c) too many cached images, (d) weak internet    |
| 3. Step-by-step solution | Step 1: Check storage → Step 2: Update the app → Step 3: Clear cached media → Step 4: Restart phone                |
| 4. Evaluate              | Did the freezing stop? If not, which cause haven't you tested yet? Repeat from Step 2 with the next possible cause |

**What just happened?** A frustrating, vague complaint ("it keeps freezing") became a testable, structured troubleshooting process — the same logic real tech support and QA engineers use every day.

### Interactive Stop-Point

**Grab a Partner:** Partner A describes a real problem they're currently facing (schoolwork, a hobby, a daily routine issue). Partner B guides them through all four stages out loud: identify, decompose, create steps, evaluate. Swap roles.

### Quick Recap

Solving a problem with computational thinking means clearly identifying it, decomposing it into smaller parts, building an ordered step-by-step algorithm, and then testing and improving that solution.

---

## 2.3 Logic as a Knowledge Representation Framework

### The Hook (Story Mode)

Alan Turing — the mathematician often called the father of computer science — spent much of his career asking a bold question: could a machine reason the way humans do? His famous "Turing Test" imagined a machine convincing a human, through pure conversation, that it could think. But underneath any "thinking" machine lies something much more basic: **logic** — the precise rules that let a system decide what's true, what's false, and what follows from what. Without logic, there is no reasoning — human or artificial.



## The Explanation

**Logic** is the formal science of correct reasoning. In computer science, logic gives us a way to represent facts and rules so that a system — human or machine — can make correct, consistent decisions.

### Logic in Computer Science

Logic helps computers understand *true* and *false* conditions, and controls how programs behave. Every **if** statement, every search filter, every security check in software is, underneath, a logical statement being evaluated as true or false.

### Role of Logic in Reasoning and Decision Making

Logic removes guesswork. Instead of "maybe" or "it depends," logic forces a decision to be based on clear facts and clear rules. This consistency is exactly why logic underlies expert systems, artificial intelligence, and even legal or medical decision-support software — fields where an incorrect or inconsistent decision could be costly or dangerous.

*Link:* Every time your streaming app decides "show this movie" or "don't show this movie" based on your age, subscription, and region, it's applying pure logic — no gut feelings involved.

## The Practical Walkthrough

Let's build a simple "if-then" statement about school life and see how logic controls a decision:

**Statement:** "If it rains, then we will stay indoors."

| Condition (input)     | Rule applied                 | Decision (output)                         |
|-----------------------|------------------------------|-------------------------------------------|
| It is raining = TRUE  | IF raining THEN stay indoors | Stay indoors                              |
| It is raining = FALSE | IF raining THEN stay indoors | Rule doesn't trigger — no forced decision |

**What just happened?** A simple English sentence became a testable rule with a clear input and a clear, predictable output — this is the exact building block every computer program is made of, repeated millions of times over.

### Interactive Stop-Point

**Pause & Think:** Write two of your own "if-then" statements about your daily school or home life (e.g., "If my alarm doesn't ring, then I will be late"). For each, identify the condition and the resulting decision.

### Quick Recap

Logic is the formal framework that lets both humans and computers represent facts and rules clearly, enabling consistent, correct decision-making instead of guesswork.

---

## 2.4 Propositional Logic

### The Hook (Story Mode)

Think about logging into a streaming platform. Behind the scenes, the system silently checks something like: *"User has the correct password AND (has an active subscription OR is on a free trial)."* If that whole statement comes out true, you get in. If it's false, you're blocked. This single sentence — built from smaller true/false pieces joined together — is a perfect real-world example of **propositional logic** in action, running behind almost every login screen you've ever used.

### The Explanation

**Propositional logic** is the branch of logic that deals with **propositions** — statements that are either true or false, and never both.

#### Proposition

A proposition is a statement with a clear, single truth value. "The sun rises in the east" is a proposition (it's true). Questions ("Is it raining?") and commands ("Close the door!") are *not* propositions — they don't have a truth value. Propositions are usually represented using capital letters like P, Q, or R.

```
Let P = "The sky is blue"
P is TRUE (on a clear day) or FALSE (on a cloudy day) – never both.
```

#### Simple and Compound Propositions

A **simple proposition** contains just one statement and cannot be broken down further — e.g., "It is raining." A **compound proposition** combines two or more simple propositions using **logical connectives** — e.g., "It is raining AND it is cold."

#### Logical Connectives

Logical connectives are the "glue" words that join propositions together. Modern computer science and mathematics use five essential connectives:

| Symbol            | Name          | Meaning                          | True when...                                              |
|-------------------|---------------|----------------------------------|-----------------------------------------------------------|
| $\wedge$          | AND           | Conjunction                      | Both statements are true                                  |
| $\vee$            | OR            | Disjunction                      | At least one statement is true                            |
| $\neg$            | NOT           | Negation                         | Reverses the truth value                                  |
| $\rightarrow$     | IMPLIES       | Conditional                      | False only when the first is true and the second is false |
| $\leftrightarrow$ | BICONDITIONAL | Biconditional ("if and only if") | Both statements have the <i>same</i> truth value          |

```

P AND Q (P $\wedge$ Q) --> True only if BOTH P and Q are true
P OR Q (P $\vee$ Q) --> True if P or Q is true (or both)
NOT P ($\neg$ P) --> True if P is false
P -> Q (P $\rightarrow$ Q) --> "If P then Q" – false only when P is true but Q is false
P <-> Q (P $\leftrightarrow$ Q) --> True only when P and Q share the same truth value

```

*Reassurance:* These symbols ( $\wedge$ ,  $\vee$ ,  $\neg$ ,  $\rightarrow$ ,  $\leftrightarrow$ ) look intimidating at first, but they're just international shorthand — a universal, compact way of writing "and," "or," "not," "if...then," and "exactly when," so mathematicians and computer scientists worldwide can write the same idea without translation.

## The Practical Walkthrough

Let's translate a real app rule into formal propositional logic.

**English rule:** "You get free shipping if your order is over \$50 and you are a Premium Member, OR if it is your first purchase."

### Step 1 — Identify the simple propositions:

- \* Let O = "Order is over \$50"
- \* Let M = "You are a Premium Member"
- \* Let F = "It is your first purchase"

### Step 2 — Translate connectors:

- \* "and"  $\rightarrow \wedge$
- \* "or"  $\rightarrow \vee$

### Step 3 — Build the compound proposition:

```
(O $\wedge$ M) $\vee$ F
```

#### Step 4 — Test it with a real scenario:

A user orders \$80 ( $O = \text{TRUE}$ ), is not a Premium Member ( $M = \text{FALSE}$ ), and it's their 5th order ( $F = \text{FALSE}$ ).

$(\text{TRUE} \wedge \text{FALSE}) \vee \text{FALSE} = \text{FALSE} \vee \text{FALSE} = \text{FALSE} \rightarrow$  No free shipping.

**What just happened?** A slightly confusing marketing sentence became an exact, testable formula — exactly what a developer would type directly into the app's checkout code.

### Interactive Stop-Point

**Pause & Think:** Translate this real-world condition into a symbolic proposition using P, Q, R and the correct connectives: *"You get free shipping if your order is over \$50 and you are a Premium Member, OR if it is your first purchase."* (Try building it yourself before checking the walkthrough above.)

### Quick Recap

Propositional logic represents statements as true or false, and combines simple propositions into compound ones using AND ( $\wedge$ ), OR ( $\vee$ ), NOT ( $\neg$ ), IMPLIES ( $\rightarrow$ ), and BICONDITIONAL ( $\leftrightarrow$ ) — the exact building blocks behind every `if` condition in software.

---

## 2.5 Truth Tables

### The Hook (Story Mode)

Imagine you're a digital circuit designer at a chip manufacturing company. Before physically building a single transistor, you need to be **100% certain** your logic behaves correctly in every possible scenario — not just the cases you happen to think of. A truth table is that guarantee: it forces you to check every single possible combination of inputs, leaving zero room for a hidden bug.

### The Explanation

A **truth table** is a table that shows the truth value (True or False) of a logical expression for every possible combination of its input propositions. If an expression has  $n$  propositions, the truth table will always have  **$2^n$  rows** — because each proposition can independently be True or False.

### Purpose of Truth Tables

Truth tables let us see, with total certainty, exactly when an expression is true and when it's false. They remove all ambiguity, help compare different logical statements, and are essential for verifying that logic in software (or hardware) behaves correctly in every situation.

## Creating Truth Tables for Logical Expressions

1. Identify all the propositions involved (e.g., P and Q).
2. List every possible combination of True/False for those propositions.
3. Apply the logical connectives step by step, building up sub-expressions as needed.
4. Write the final result for each row.

## Truth Values of Propositions

Each proposition can independently be True (T) or False (F). With 2 propositions, there are  $2^2 = 4$  rows. With 3 propositions, there are  $2^3 = 8$  rows, and so on.

## The Practical Walkthrough: Building a Full Truth Table for AND, OR, and NOT

| P | Q | $P \wedge Q$ (AND) | $P \vee Q$ (OR) | $\neg P$ (NOT P) |
|---|---|--------------------|-----------------|------------------|
| T | T | T                  | T               | F                |
| T | F | F                  | T               | F                |
| F | T | F                  | T               | T                |
| F | F | F                  | F               | T                |

Reading this table:

- \*  $P \wedge Q$  is only True in row 1, where *both* P and Q are True.
- \*  $P \vee Q$  is True in every row except the last, where both are False.
- \*  $\neg P$  simply flips P's value in every row.

## The Practical Walkthrough: Truth Table for IMPLIES ( $\rightarrow$ ) and BICONDITIONAL ( $\leftrightarrow$ )

| P | Q | $P \rightarrow Q$ | $P \leftrightarrow Q$ |
|---|---|-------------------|-----------------------|
| T | T | T                 | T                     |
| T | F | F                 | F                     |
| F | T | T                 | F                     |
| F | F | T                 | T                     |

**What just happened?** Notice something students often find surprising:  $P \rightarrow Q$  is *True* whenever P is False, no matter what Q is! In plain English: "If P is false, the promise 'if P then Q' can't be broken — it's automatically considered kept." (Think of it like a promise: "If it rains, I'll bring an umbrella." If it *doesn't* rain, you haven't broken your promise either way.)

## Interactive Stop-Point

**Grab a Partner:** Partner A writes a mystery compound proposition using two variables (e.g.,  $\neg P \vee Q$ , or  $(P \wedge Q) \rightarrow P$ ). Partner B builds the complete truth table for it and determines whether it's a **Tautology** (always True), a **Contradiction** (always False), or a **Contingency** (True in some rows, False in others). Swap and repeat.

## Quick Recap

A truth table lists every possible input combination for a logical expression and shows the resulting output for each, giving total certainty about when an expression is true — with  $2^n$  rows for  $n$  propositions.

---

## 2.6 Propositional Equivalence

### The Hook (Story Mode)

In the 1840s, mathematician Augustus De Morgan discovered something that would, over a century later, save chip manufacturers millions of transistors: he proved that negating an AND statement is logically the *same* as an OR statement of the negations — and vice versa. Today, every time a circuit designer simplifies a digital chip's logic using **De Morgan's Laws**, they're using a 180-year-old mathematical trick to make phones faster, cheaper, and more power-efficient.

### The Explanation

**Propositional equivalence** means two logical expressions — even if they *look* completely different — always produce the exact same truth value, in every single row of their truth tables. The symbol  $\equiv$  is used to show equivalence.

### Identifying Equivalent Logical Expressions

To check if two expressions are equivalent, build a truth table for each and compare the final output columns row by row. If they match in every row, the expressions are equivalent.

### Importance of Equivalence in Logic

Equivalence lets us simplify complex expressions into simpler, easier-to-read (or easier-to-compute) ones without changing their meaning. This saves time, improves code efficiency, and prevents logical errors.

### De Morgan's Laws

These are two of the most important equivalence rules in all of computer science:

$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$  ("NOT (P AND Q)" is the same as "(NOT P) OR (NOT Q)")  
 $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$  ("NOT (P OR Q)" is the same as "(NOT P) AND (NOT Q)")

*Link:* Imagine a security rule: "Access denied unless (username is valid AND password is valid)." Using De Morgan's Law, "NOT (valid username AND valid password)" is logically identical to "invalid username OR invalid password" — the exact same access check, written differently, and sometimes easier for a programmer to implement one way over the other.

## Tautologies and Contradictions

- A **tautology** is an expression that is **always True**, no matter the input values (e.g.,  $P \vee \neg P$  — "P or not P" is always true).
- A **contradiction** is an expression that is **always False**, no matter the input values (e.g.,  $P \wedge \neg P$  — "P and not P" can never be true).
- A **contingency** is an expression that is sometimes True and sometimes False, depending on the inputs — most everyday expressions fall into this category.

## The Practical Walkthrough: Proving De Morgan's Law with a Truth Table

Let's prove  $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$  using a complete 4-row truth table.

| P | Q | $P \wedge Q$ | $\neg(P \wedge Q)$ | $\neg P$ | $\neg Q$ | $\neg P \vee \neg Q$ |
|---|---|--------------|--------------------|----------|----------|----------------------|
| T | T | T            | F                  | F        | F        | F                    |
| T | F | F            | T                  | F        | T        | T                    |
| F | T | F            | T                  | T        | F        | T                    |
| F | F | F            | T                  | T        | T        | T                    |

**What just happened?** Compare the two bolded columns —  $\neg(P \wedge Q)$  and  $\neg P \vee \neg Q$  — row by row. They match perfectly in all 4 rows. That's a formal, airtight proof that these two expressions are logically equivalent, even though they're written completely differently.

## Interactive Stop-Point

**Pause & Think:** Using the same method, try proving the second De Morgan's Law,

$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$ , by building your own 4-row truth table. Do the final columns match?

## Quick Recap

Propositional equivalence means two differently-written expressions always share the same truth values — proven using truth tables — and De Morgan's Laws are the most important equivalence rules, letting AND/OR statements be rewritten as their negated opposites.

## 2.7 Propositional Satisfiability

### The Hook (Story Mode)

Every time you board a plane, its autopilot software has been checked by a category of tool called a **SAT solver** — a program built entirely on the concept of **satisfiability**. SAT solvers can take millions of logical constraints (safety rules, hardware limits, flight conditions) and determine: is there *any* possible combination of values that satisfies all of them at once, without contradiction? This exact idea — tracing back to centuries-old logic puzzles like the Knight's Tour — now keeps modern airplanes, chip designs, and security software provably correct.

### The Explanation

**Propositional satisfiability** asks a simple but powerful question: **is there at least one combination of truth values that makes this expression True?**

- If **yes** — at least one True outcome exists — the expression is **satisfiable**.
- If **no** — every possible combination results in False — the expression is **unsatisfiable** (a contradiction).

### Conditions for a Proposition to be Satisfiable

A proposition is satisfiable if, when you check every row of its truth table, at least one row results in True. This depends entirely on the connectives used and how the propositions are combined.

### Examples of Satisfiable and Unsatisfiable Expressions

- $P \vee Q$  — **Satisfiable**. True whenever P or Q (or both) are true.
- $P \wedge Q$  — **Satisfiable**. True when both P and Q are true.
- $P \wedge \neg P$  — **Unsatisfiable**. P can never be simultaneously true and false — every row results in False.

### The SAT Problem

Determining whether a complex logical expression (with potentially thousands of variables) is satisfiable is called the **SAT problem** — one of the most important problems in all of computer science. It's used to verify software correctness, design digital circuits, solve scheduling puzzles, and even crack certain types of security codes.

### The Practical Walkthrough: Testing Satisfiability of $(P \vee Q) \wedge (\neg P \vee \neg Q)$

**Step 1 — List the propositions:** P and Q.

**Step 2 — Build the truth table, evaluating sub-expressions first:**



| P | Q | $P \vee Q$ | $\neg P$ | $\neg Q$ | $\neg P \vee \neg Q$ | $(P \vee Q) \wedge (\neg P \vee \neg Q)$ |
|---|---|------------|----------|----------|----------------------|------------------------------------------|
| T | T | T          | F        | F        | F                    | F                                        |
| T | F | T          | F        | T        | T                    | T                                        |
| F | T | T          | T        | F        | T                    | T                                        |
| F | F | F          | T        | T        | T                    | F                                        |

### Step 3 — Check the final column for at least one True.

Rows 2 and 3 both result in True.

**Conclusion:** The expression  $(P \vee Q) \wedge (\neg P \vee \neg Q)$  is **satisfiable** — in fact, this expression is a famous one: it's logically equivalent to **XOR** ("exclusive or" — true when exactly one of P, Q is true, but not both).

**What just happened?** Instead of guessing, you methodically tested all 4 possible input combinations and found concrete proof — 2 out of 4 rows — that a satisfying assignment exists.

### Interactive Stop-Point

**Pause & Think:** A food delivery app has this rule: "Bring pizza OR (burger AND do NOT bring soda)." Is this expression satisfiable? Find one combination of choices that makes it True. Then try the rule: "You must bring pizza AND you must NOT bring pizza." Is this one satisfiable? Why or why not?

### Quick Recap

An expression is satisfiable if at least one combination of truth values makes it True, and unsatisfiable if no such combination exists — a concept at the heart of modern SAT solvers that verify software, chip designs, and safety-critical systems.

## 2.8 Predicate Logic

### The Hook (Story Mode)

Propositional logic is powerful, but it has a serious limitation: it can only handle whole, fixed statements — it can't "look inside" a sentence. Consider the proposition  $A = \text{"Ali is tall."}$

Propositional logic treats this as one indivisible block: A is either True or False, full stop. It can't separate the *subject* (Ali) from the *property* (tall). **Predicate logic** was developed precisely to fix this — allowing logic to reach inside a sentence and reason about individual objects, their

properties, and their relationships to each other. This upgrade is what allows modern databases, search engines, and AI systems to reason about millions of individual entities, not just fixed yes/no facts.

## The Explanation

**Predicate logic** (also called first-order logic) extends propositional logic by introducing **variables** and **predicates**, allowing us to express detailed statements about objects and their properties.

### Introduction to Predicate Logic

A **predicate** is a statement whose truth depends on the value of one or more variables. For example, `IsEven(x)` is true or false *depending on what x is*: `IsEven(4)` is True, but `IsEven(7)` is False. Predicate logic lets us express general statements like "All students are hardworking" or "Some books are interesting" — statements propositional logic simply cannot capture precisely.

### Difference Between Propositional and Predicate Logic

| Feature                             | Propositional Logic                    | Predicate Logic                                       |
|-------------------------------------|----------------------------------------|-------------------------------------------------------|
| Basic unit                          | Whole fixed statements (P, Q)          | Predicates with variables, e.g., <code>Tall(x)</code> |
| Can see inside a statement?         | No — treats "Ali is tall" as one block | Yes — separates subject (Ali) from property (Tall)    |
| Expressiveness                      | Limited                                | Much more expressive and flexible                     |
| Example                             | A = "Ali is Tall"                      | <code>Tall(Ali)</code>                                |
| Can generalize across many objects? | No                                     | Yes, using quantifiers (see 2.9)                      |

Propositional Logic:  
A = "Ali is Tall"  
Cannot see inside A  
Limited

Predicate Logic:  
`Tall(Ali)`  
Can see subject (Ali) and property (Tall)  
Much more expressive

### Predicates and Variables

- A **predicate** describes a property or relationship — e.g., `IsEven(x)`, `Tall(x)`, `Teaches(t, s)`.
- A **variable** (like x, y) represents an unspecified object from a domain (a set of possible values) — e.g., "For all x, x > 0" uses x as a variable ranging over numbers.

*Link:* Every time a database query says "find all users WHERE age > 18," it's using a predicate (`age(x) > 18`) applied across every row (every value of  $x$ ) in the database — this *is* predicate logic, running behind the scenes of nearly every app you use.

## The Practical Walkthrough: Translating Predicates

Let's translate three real statements into formal predicate logic.

| English statement                 | Predicate logic translation            |
|-----------------------------------|----------------------------------------|
| "x is greater than 5"             | <code>GreaterThan(x, 5)</code>         |
| "Ali borrows a book"              | <code>Borrows(Ali, Book)</code>        |
| "The teacher teaches the student" | <code>Teaches(teacher, student)</code> |

**What just happened?** Each English sentence was broken into a predicate name (the relationship or property) and its arguments (the specific objects involved) — this exact translation process is step one of writing any database query or knowledge-base rule.

## Interactive Stop-Point

**Pause & Think:** Convert this sentence into predicate logic: "The library has the book available."  
(Hint: identify the predicate name and its argument(s), similar to the examples above.)

## Quick Recap

Predicate logic upgrades propositional logic by using predicates and variables, letting us reach inside statements to describe properties and relationships between specific objects, rather than treating whole sentences as fixed, unbreakable blocks.

---

## 2.9 Quantifiers in Predicate Logic

### The Hook (Story Mode)

Compare these two sentences: "*Every student passed the exam*" versus "*At least one student got a perfect score*." They sound similar in structure, but they make completely different claims — one is about **everyone**, the other about **someone**. In predicate logic, this exact distinction is captured with two small but powerful symbols:  $\forall$  (for all) and  $\exists$  (there exists). Getting this distinction right is often the difference between a database query that works correctly and one that silently returns the wrong result.

## The Explanation

**Quantifiers** describe *how many* objects in a domain satisfy a given predicate. They turn a predicate like `Tall(x)` — which alone has no fixed truth value — into a complete, testable statement.

### Universal Quantifier ( $\forall$ — "For All")

The universal quantifier states that a predicate is true for **every** object in the domain. Written  `$\forall x, P(x)$` , meaning "For all x, P(x) is true."

```
 $\forall x, x > 0$ --> "For all x, x is greater than 0"
 $\forall x, \text{Student}(x) \rightarrow \text{HasID}(x)$ --> "Every student has an ID"
```

### Existential Quantifier ( $\exists$ — "There Exists")

The existential quantifier states that a predicate is true for **at least one** object in the domain. Written  `$\exists x, P(x)$` , meaning "There exists at least one x such that P(x) is true."

```
 $\exists x, x \text{ is even}$ --> "There exists at least one x that is even"
 $\exists x, \text{Book}(x) \wedge \text{Available}(x)$ --> "There exists at least one book that is available"
```

## Using Quantifiers in Logical Statements

Quantifiers can be combined for more complex statements. Consider:

```
 $\forall x \exists y, x < y$ --> "For every x, there exists a y that is greater than x"
```

*Reassurance:*  $\forall$  and  $\exists$  are just shorthand for two everyday English phrases — "every single one" and "at least one." Once you see them that way, the symbols stop being scary.

## The Practical Walkthrough: Quantifier Order Matters!

Here's one of the trickiest — and most important — lessons in predicate logic: **the order of quantifiers changes the meaning entirely.**

Consider the predicate `Loves(x, y)` meaning "x loves y," over the domain of all people.

| Statement   | Symbolic form                                                     | Plain-English meaning                                                                                                                       |
|-------------|-------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Statement A | <code><math>\forall x \exists y, \text{Loves}(x, y)</math></code> | "For every person x, there exists <i>some</i> person y that x loves" → <b>Everyone loves someone</b> (but not necessarily the same someone) |
| Statement B | <code><math>\exists y \forall x, \text{Loves}(x, y)</math></code> | "There exists <i>one</i> person y that every person x loves" → <b>There is one specific person loved by everyone</b>                        |

### Step-by-step comparison:

1. Statement A fixes  $x$  first, then searches for *any*  $y$  that makes it true — the  $y$  can be *different* for each  $x$ .
2. Statement B fixes  $y$  first, and that *same*  $y$  must work for *every*  $x$  — a much stronger, more specific claim.
3. Statement B implies Statement A (if everyone loves one specific person, then certainly everyone loves *someone*) — but Statement A does **not** imply Statement B (everyone loving *someone* doesn't mean they all love the *same* someone).

**What just happened?** Two nearly identical-looking symbolic statements — just with  $\forall$  and  $\exists$  swapped in order — turned out to mean two completely different real-world claims. This is exactly why precision matters so much in formal logic.

### Interactive Stop-Point

**Pause & Think:** What is the difference between  $\forall x \exists y, \text{Loves}(x, y)$  and  $\exists y \forall x, \text{Loves}(x, y)$ ? Does everyone love someone (possibly different people), or is there one specific person loved by everyone? Try inventing your own real-world example (not about love) that shows the same distinction — for instance, about students and teachers, or customers and products.

### Quick Recap

The universal quantifier ( $\forall$ ) means "true for every object," while the existential quantifier ( $\exists$ ) means "true for at least one object" — and critically, swapping their order in a statement can completely change its meaning.

---

## 2.10 Applying Predicate Logic to Real-World Problems

### The Hook (Story Mode)

Predicate logic isn't just an abstract math exercise — it's the invisible engine behind family-tree apps, social network "people you may know" suggestions, and even the access-control systems that decide who can open which door in a secure building. Anywhere you have objects (people, books, files) and relationships between them, predicate logic is quietly doing the reasoning.

### The Explanation

Applying predicate logic to real problems means modeling real objects, their properties, and their relationships using predicates and quantifiers — then using that model to reason and draw conclusions.

## Representing Relationships Using Predicates

Real-world relationships translate naturally into predicates with multiple arguments:

```
Borrows(x, y) --> "student x borrows book y"
Teaches(t, s) --> "teacher t teaches student s"
```

These predicates let us model an entire system — like a school library — in a structured, queryable way.

## Writing Logical Statements With Quantifiers

Combining predicates with quantifiers lets us write general rules about a whole system:

```
∀x, Student(x) → Borrows(x, Book)
--> "Every student borrows a book"

∃y, Book(y) ∧ Available(y)
--> "There exists at least one book that is available"
```

## Reasoning and Drawing Conclusions

Once we have facts and general rules, predicate logic lets us reason toward specific conclusions. For example, given:

- **Rule:**  $\forall x, \text{Student}(x) \rightarrow \text{HasLibraryCard}(x)$  ("Every student has a library card")
- **Fact:**  $\text{Student}(\text{Ali})$  ("Ali is a student")

We can logically conclude:  $\text{HasLibraryCard}(\text{Ali})$  ("Ali has a library card").

*Link:* This exact reasoning pattern — a general rule plus a specific fact producing a specific conclusion — is how automated systems check eligibility: "every Premium user gets free shipping" + "Ali is a Premium user" → "Ali gets free shipping," verified instantly and consistently, without a human needing to check manually.

## The Practical Walkthrough: Modeling a Library System

### Step 1 — Define the predicates:

- \*  $\text{Student}(x)$  — x is a student
- \*  $\text{Book}(y)$  — y is a book
- \*  $\text{Borrows}(x, y)$  — x borrows y
- \*  $\text{Available}(y)$  — y is available

### Step 2 — Write the general rules:

```
Rule 1: $\forall x, \text{Student}(x) \rightarrow \text{HasLibraryCard}(x)$
Rule 2: $\exists y, \text{Book}(y) \wedge \text{Available}(y)$
```

### Step 3 — Add specific facts:

Fact 1: `Student(Ali)`

### Step 4 — Apply reasoning:

Using Rule 1 and Fact 1 together: since Ali is a student, and every student has a library card, we conclude `HasLibraryCard(Ali)`.

**What just happened?** You just built — by hand — the exact type of small "knowledge base" that powers real library management software, automatically checking permissions and rules without a human re-checking each case individually.

## Interactive Stop-Point

**Pause & Think:** Using the predicates above, write a rule (in predicate logic) for the statement: "All students must attend class." Then write a specific fact about one student, and state what conclusion could be logically drawn by combining the rule and the fact.

## Quick Recap

Predicate logic models real-world systems by defining predicates for objects and relationships, writing general rules with quantifiers, and then combining those rules with specific facts to reach reliable, automatic conclusions.

---

## 2.11 Logic-Based Reasoning and Inference

### The Hook (Story Mode)

Picture a smart home security system at 2 AM: *"IF motion is detected at night, THEN sound the alarm. Motion was detected at night. THEREFORE, sound the alarm."* This tiny three-line reasoning chain is a real, formal logical proof — and it's running, in some form, inside every smart alarm, fraud-detection system, and automated decision engine in the world today.

### The Explanation

**Logic-based reasoning and inference** is the process of deriving new, guaranteed-true facts from known information, using strict logical rules.

### Logical Inference

Inference means reaching a new conclusion from existing facts, using valid logical steps. If the starting facts (called **premises**) are true, and the reasoning steps are valid, the conclusion is **guaranteed** to be true — not just probably true.

Classic example:

```
Premise 1: All humans are mortal.
Premise 2: Socrates is a human.
Conclusion: Socrates is mortal.
```

## Deduction Using Logical Rules

**Deduction** reasons from general rules down to a specific conclusion. Several formal deduction patterns are especially important in computer science:

**Modus Ponens** ("the way that affirms"):

```
Premise 1: $P \rightarrow Q$ ("If P then Q")
Premise 2: P ("P is true")
Conclusion: Q ("Therefore, Q is true")
```

**Modus Tollens** ("the way that denies"):

```
Premise 1: $P \rightarrow Q$ ("If P then Q")
Premise 2: $\neg Q$ ("Q is false")
Conclusion: $\neg P$ ("Therefore, P is false")
```

**Syllogism** (chaining two conditional rules together):

```
Premise 1: $P \rightarrow Q$
Premise 2: $Q \rightarrow R$
Conclusion: $P \rightarrow R$
```

*Link:* Modus Ponens is the direct engine behind rule-based expert systems and smart-home automation ("if motion detected, then alarm"). Modus Tollens is the logic behind troubleshooting by elimination ("if the bulb worked, the room would be lit; the room isn't lit; therefore, the bulb didn't work").

## The Practical Walkthrough: A Security Lockout Decision Using Deduction

Let's trace a real system decision step by step, using Modus Tollens.

**Scenario:** A company's login system has this rule: "If a user enters the correct password (P), then they are granted access (Q)." A specific user was **not** granted access.

**Step 1 — Write the general rule:**

```
 $P \rightarrow Q$ ("If correct password entered, then access granted")
```

**Step 2 — State the observed fact:**



$\neg Q$  ("Access was NOT granted")

### Step 3 — Apply Modus Tollens:

$P \rightarrow Q$

$\neg Q$

---

$\therefore \neg P$  ("Therefore, the correct password was NOT entered")

### Step 4 — Interpret the conclusion:

The system can now confidently and automatically log: "Incorrect password attempt" — and, if this repeats several times, trigger an automated security lockdown.

**What just happened?** Without any human needing to manually inspect the situation, a purely logical deduction — Modus Tollens — allowed the system to correctly and reliably identify *why* access was denied, directly from one rule and one observation.

## Interactive Stop-Point

**Grab a Partner:** Partner A writes a real or invented "if-then" rule about school or home life (e.g., "If the fire alarm rings, then everyone must exit the building"). Partner B is given either the "if" fact or the negation of the "then" fact, and must correctly apply Modus Ponens or Modus Tollens to reach the valid conclusion. Swap and repeat with a new rule.

## Quick Recap

Logical inference lets us derive new, guaranteed conclusions from known facts using rules like Modus Ponens and Modus Tollens — the exact reasoning engine behind automated systems, from smart-home alarms to security lockouts.

---

## Chapter Summary

| Concept                             | Definition                                                                                                                                |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Computational Thinking</b>       | A problem-solving approach in which a problem is understood, broken into smaller parts, and solved step by step using logical thinking.   |
| <b>Decomposition</b>                | The process of breaking a large or complex problem into smaller and manageable parts.                                                     |
| <b>Algorithm</b>                    | A step-by-step set of instructions used to solve a problem or perform a task.                                                             |
| <b>Logic</b>                        | A formal method of representing facts, statements, and rules so that correct reasoning and decision-making can be performed.              |
| <b>Proposition</b>                  | A statement that is either true or false, but not both at the same time.                                                                  |
| <b>Compound Proposition</b>         | A proposition formed by combining two or more simple propositions using logical operators such as AND, OR, and NOT.                       |
| <b>Truth Table</b>                  | A table used to show the truth values of a logical expression for all possible combinations of its inputs.                                |
| <b>Propositional Equivalence</b>    | Two logical expressions have the same truth values in all possible cases.                                                                 |
| <b>Propositional Satisfiability</b> | A proposition is satisfiable if there is at least one condition under which it becomes true.                                              |
| <b>Predicate Logic</b>              | A type of logic that represents statements using subjects, properties, variables, and relationships.                                      |
| <b>Predicate</b>                    | A logical expression that describes a property or relationship of a subject, such as <code>Tall(Ali)</code> .                             |
| <b>Quantifier</b>                   | Used in predicate logic to show how many objects a statement applies to, such as "for all" ( $\forall$ ) or "there exists" ( $\exists$ ). |
| <b>Logical Inference</b>            | The process of drawing a correct conclusion from given facts or statements.                                                               |
| <b>Deduction</b>                    | A reasoning method in which a specific conclusion is derived from general rules or known facts.                                           |

# End-of-Chapter Exercise

## Multiple Choice Questions

1. Computational thinking is mainly used for:  
(a) Playing computer games (b) Solving problems in a logical way (c) Writing essays (d) Drawing pictures
2. The first step in problem solving is:  
(a) Writing code (b) Evaluating the solution (c) Identifying the problem (d) Breaking the problem
3. Breaking a large problem into smaller parts is called:  
(a) Abstraction (b) Decomposition (c) Evaluation (d) Automation
4. A step-by-step solution to a problem is known as an:  
(a) Program (b) Flowchart (c) Algorithm (d) Function
5. The logic that uses only true or false values is:  
(a) Predicate logic (b) Fuzzy logic (c) Propositional logic (d) Natural logic
6. A compound proposition is formed using:  
(a) A single statement (b) Only numbers (c)  $(P \wedge Q)$  (d) Only variables
7. The purpose of a truth table is to:  
(a) Write programs (b) Store data (c) Evaluate logical expressions (d) Design networks
8. A proposition that is true for at least one case is called:  
(a) Unsatisfiable (b) Invalid (c) Satisfiable (d) Equivalent
9. The symbol for the universal quantifier is:  
(a)  $\exists$  (b)  $\wedge$  (c)  $\forall$  (d)  $\neg$
10. Predicate logic is mainly used to:  
(a) Draw diagrams (b) Represent relationships using variables (c) Store numbers (d) Design networks

**Answer Key:** 1-b, 2-c, 3-b, 4-c, 5-c, 6-c, 7-c, 8-c, 9-c, 10-b

## Short Questions

1. What is meant by computational thinking?
2. Why is computational thinking important in problem solving?
3. What is decomposition in problem solving?
4. What is an algorithm?
5. What is logic in computer science?
6. What is a proposition?

7. What are truth values?
8. What is a truth table?
9. What is propositional satisfiability?
10. What is a predicate in predicate logic?
11. What is the difference between the universal and existential quantifiers?
12. What is logical inference?

## Long Questions

1. Explain computational thinking and discuss its importance in solving problems, with suitable examples.
2. Describe the steps involved in problem solving using computational thinking. Explain decomposition with an example.
3. What is logic as a knowledge representation framework? Explain its role in reasoning and decision making.
4. Explain propositional logic. Discuss simple and compound propositions with examples.
5. What are truth tables? Explain how truth tables are created and used to evaluate logical expressions.
6. Explain propositional equivalence. Describe how equivalent logical expressions are identified, and explain De Morgan's Laws with a truth table proof.
7. What is propositional satisfiability? Explain satisfiable and unsatisfiable propositions with examples.
8. Explain predicate logic and quantifiers. Discuss how predicate logic is applied to real-world problems.
9. Explain logical inference and deduction. Describe Modus Ponens and Modus Tollens with examples.

---

*End of Chapter 2: Computational Thinking & Algorithms.*